ARE SELF-EVOLVING AGENT SKILLS LEARNING SKILLS?

Anonymous Submission

ABSTRACT

Self-evolving agents store experience in persistent skills, but a higher task score leaves open what useful behavior has become reusable. We diagnose completed SkillEvolver, SPARK, and CoEvoSkills artifacts through repeated paired evaluations and controlled task variations. Our analysis separates recurring aggregate utility from agreement on individual tasks, and decomposes the net gain into improvements and losses. We further distinguish gains on zero-reward tasks from interference with full-reward executions. All three methods improve source-task means in both runs, while task changes expose different scopes of utility. SkillEvolver sustains positive gains across the tested changes by combining more frequent improvements with fewer losses of existing success. These results characterize skill learning through the persistence and distribution of useful behavior beyond the source task.

***Index Terms*—** Agent skills, self-evolution, diagnostic evaluation, skill reuse, generalization

1. INTRODUCTION

Persistent agent skills offer a route to improvement without updating model parameters. Instructions, scripts, and supporting references make execution experience available to later runs. SkillEvolver, SPARK, and CoEvoSkills construct these artifacts through deployment-grounded revision, verified-trajectory distillation, and co-evolutionary verification [1–3]. Their outputs motivate the question in our title: when a system produces a higher-scoring skill, what useful behavior has become persistent?

The location of that improvement matters. A skill can preserve a decision rule that remains useful on new inputs, a choice tailored to the original instance, or a check that a later executor fails to apply. These artifacts may all help on the source task, yet lead to different outcomes when the task changes. The final score also combines artifact construction, execution, and verification. Understanding skill learning therefore requires following the stored guidance into the behavior it supports.

We ask two connected questions. **Does the benefit persist, and under which task changes?** Repeated source-task evaluations test whether paired utility recurs, while frozen-skill variants expose the demands that the same guidance can accommodate. **Which task-level changes produce the apparent improvement?** A positive mean may reflect widespread gains or a small balance between large benefits and losses. Identifying these patterns distinguishes the creation of useful behavior from interference with behavior the executor already performs successfully.

We examine three construction mechanisms through two standard-task runs per method and controlled changes to input representation, task instances, and coupled requirements. Each skill condition is paired with a no-skill execution of the same task. We then match task identities across runs, separate the positive and negative contributions to variant utility, and examine score transitions from the two baseline endpoints. The completed skill in use is the common object of diagnosis.

All three methods yield positive mean source-task gains in both runs, but this agreement does not persist under task variation. SkillEvolver remains positive across all three change types. SPARK's benefit depends on the change type, and CoEvoSkills has negative mean gains in all three. The clearest separation occurs when the instance is replaced. The decomposition explains how these means arise: all three methods produce substantial gains on some variants, while their frequency of improvement and cost to existing successes differ.

Our contribution is a diagnosis of the evidence behind observed skill gains. Repeated evaluations measure agreement across runs; controlled variants locate the scope of utility; gain–loss and baseline-conditioned analyses distinguish added benefits from disruption of existing success.

2. PROBLEM FORMULATION

2.1. Skill Revision and Evaluation

At revision round r, an optimizer updates the current persistent skill artifact:

$$s_{r+1} = O(s_r, \tau_r, f_r), \tag{1}$$

where s_r is the current skill, τ_r is an execution trace, and f_r is feedback from execution and evaluation. The operator O denotes the method's complete update pathway, including its native merge, validation, selection, or adoption mechanism. A generated candidate that is rejected or never adopted is not s_{r+1}.

The agent then executes task x with the frozen skill:

$$o_r = A(x, s_r, z_r), \tag{2}$$

where z_r represents model sampling, tool use, and environment state, and o_r is the resulting output artifact or state change. A verifier maps the output to a score:

$$y_r = V(x, o_r). \tag{3}$$

自进化智能体技能是否在学习技能?

匿名提交

摘要

自进化智能体将经验存储在持久技能中，但任务得分提高后，哪些有用行为能够复用仍有待解释。我们通过重复配对评估与受控任务变化，诊断 SkillEvolver、SPARK 和 CoEvoSkills 的最终技能。分析区分总体效用的重复出现与逐题结果的一致性，并将净增益分解为改进和损失；进一步区分零分任务上的收益与满分执行上的损失。三种方法两次均提高源任务均分，任务变化却揭示了不同的有效范围。SkillEvolver 通过更频繁的改进和更少的原有成功损失，在各类受测变化下保持正向收益。这些结果从有用行为在源任务之外的持续性及分布刻画技能学习。

关键词—智能体技能，自进化，诊断评估，技能复用，泛化

1. 引言

持久技能为智能体提供了一条无需更新模型参数的改进路径。指令、脚本和参考资料让后续执行能够利用已有经验。SkillEvolver、SPARK 和 CoEvoSkills 分别通过基于部署的修订、经验证轨迹的蒸馏和协同进化验证来构建技能产物 [1–3]。这些产物引出了标题中的问题：系统生成得分更高的技能时，哪些有用行为得以持久保留?

那种改进的位置很重要。一个技能可以保留一个在新的输入上仍然有用的决策规则、一个针对原始实例的选择，或一个后续执行者未能应用的检查。这些工件可能在源任务上有所帮助，但在任务变化时会导致不同的结果。最终得分还结合了工件构建、执行和验证。因此，理解技能学习需要跟随存储的指导进入它支持的行为。

我们提出两个相关问题。收益能否持续，在哪些任务变化下持续？重复源任务评估检验配对效用是否反复出现，冻结技能的变体评估揭示同一指导能够满足哪些需求。哪些逐题变化构成了观察到的提升？正向均值可能来自广泛的收益，也可能只是大量收益与损失抵消后的微小差额。识别这些模式，可以区分新增的有用行为与对执行者原有成功行为的干扰。

我们通过每个方法的标准任务运行两次和控制输入表示、任务实例以及耦合需求的变化，来检验三种构建机制。每个技能条件都与相同任务的无技能执行配对。然后，我们在运行之间匹配任务身份，分离对变体效用的正面和负面贡献，并检验从两个基线端点的分数转换。正在使用的完整技能是诊断的共同对象。

三种方法在两次运行中均有正向源任务平均增益，但这种一致性在任务变化后不再成立。SkillEvolver 在三类变化下均为正；SPARK 的收益取决于变化类型；CoEvoSkills 在三类变化下的平均增益均为负。实例替换时分化最明显。分解分析揭示了这些均值的组成：三种方法都能在部分变体上带来较大增益，但改进频率和原有成功受到的损失不同。

我们的贡献是对观察到的技能提升背后证据的诊断。重复评估测量跨运行的一致性；受控变体定位实用范围；收益-损失和基线条件分析区分了新增效益与现有成功模式的干扰。

2. 问题表述

2.1. 技能修订与评估

在修订轮次 r，优化器更新当前持久的技能工件：

$$s_{r+1} = O(s_r, \tau_r, f_r), \tag{1}$$

其中 s_r 是当前技能， τ_r 是执行轨迹，而 f_r 是来自执行和评估的反馈。运算符 O 表示方法完整的更新路径，包括其原生合并、验证、选择或采纳机制。一个被拒绝或从未被采纳的生成候选者不是 s_{r+1}。

随后，智能体使用冻结技能执行任务 x：

$$o_r = A(x, s_r, z_r), \tag{2}$$

z_r 表示模型采样、工具使用和环境状态，而 o_r 是产生的输出工件或状态变化。验证器将输出映射到分数：

$$y_r = V(x, o_r). \tag{3}$$

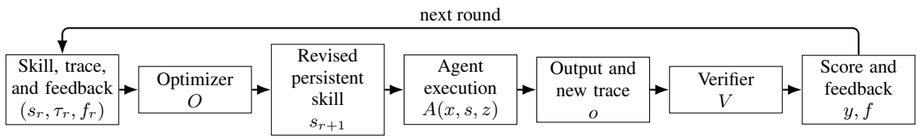


Fig. 1. The skill-evolution lifecycle. Construction produces a persistent artifact; deployment turns its guidance into behavior; verification assigns the observed score. Our evaluation measures completed-artifact utility through paired outcomes, repeated runs, and frozen-skill task variation.

2.2. From Score Gains to Skill Behavior

The score $V(x, A(x, s, z))$ records the outcome of a skill used by an executor and assessed by a verifier. Comparing a completed skill with no skill measures its utility at deployment. Comparing the adopted final skill with its actual parent addresses the improvement associated with revision. The experiments here follow the first contrast across repeated runs and changed tasks.

For each source task x, we freeze its completed skill s_x and evaluate it on a variant $T_k(x)$ alongside a no-skill execution. The skill remains unchanged, while the executor can reinterpret the input, recompute a solution, and adapt its actions. A positive paired difference on the variant indicates that the stored guidance remains useful under that change. The variation types distinguish the decisions that can be reused from those that the executor must reconsider.

To interpret the observed utility, we examine its distribution over tasks. Agreement between runs describes whether the same tasks benefit repeatedly. Separating positive and negative score changes reveals whether a small mean reflects small effects or cancellation. Conditioning on the no-skill score distinguishes help on zero-reward tasks from harm to full-reward executions. These measurements retain the paired task as the unit of analysis while answering different questions about the behavior associated with the skill.

3. DOES SKILL UTILITY PERSIST BEYOND THE SOURCE TASK?

We first test whether completed skills provide a recurring benefit on their source tasks. We then change the task while keeping its skill frozen to determine which demands that benefit survives. The paired no-skill execution provides the reference for both comparisons.

3.1. Experimental Setup

We evaluate SkillEvolver [1], SPARK [2], and CoEvoSkills [3] on SkillsBench [4], with two runs per method. Each run pairs the method's skill condition with a no-skill execution of the same task. Both runs of each method use the same task set: 81 tasks for SkillEvolver, 84 for SPARK, and 85 for CoEvoSkills.

We report mean verifier reward on a 0–100 scale and the mean within-task difference between the skill and no-skill conditions. For a run with N tasks, the paired gain is

$$\Delta = \frac{100}{N} \sum_{i=1}^{N} \left(r_i^{\text{skill}} - r_i^{\text{no-skill}} \right). \tag{4}$$

We also count tasks with positive, negative, and zero differences.

3.2. Positive Source-Task Utility in Both Runs

All three methods have positive mean paired gains in both runs (Table 1). SkillEvolver gains 25.61 and 25.02 points, CoEvoSkills gains 10.24 and 9.76, and SPARK gains 1.84 and 6.45. SkillEvolver and CoEvoSkills retain similar gains despite substantial changes in their no-skill scores. The paired comparison reveals recurring utility that absolute skill-condition scores alone would obscure.

The task-level results also differ. SkillEvolver has at least four times as many improvements as regressions in each run; CoEvoSkills improves 17 tasks in each run, with nine and ten regressions. SPARK's first run is nearly balanced, while its second has 13 improvements and five regressions. The gain therefore reflects both the size and frequency of task-level changes. The frozen-skill variants test whether the useful guidance behind these source-task outcomes survives a different task demand.

3.3. Task Variation Tests What Remains Useful

We apply each source-task skill to controlled variants without further evolution. Input-representation changes (P1) alter names, column arrangements, or schemas; task-instance changes (P2) replace the data or instance structure; coupled-requirement changes (P3) alter or add requirements that must be satisfied together. The corresponding decision demands are to remap the input, recompute a solution for a replacement instance, and accommodate changed correctness requirements. A frozen skill must support these adaptations through guidance already present in the artifact.

The complete record contains 222 variants and 886 condition-level results across no skill and the three methods. Table 1 summarizes historical and recovery evaluations for 222 SkillEvolver variants and 221 each for SPARK and CoEvoSkills, pairing each skill condition with the no-skill result on the same variant.

Source-task improvement does not imply useful generalization. All three methods improve mean reward on the source tasks in both runs, but their variant results diverge. SkillEvolver retains a positive mean gain of 11.94 points, SPARK is close to neutral at $+0.65$, and CoEvoSkills falls below the no-skill baseline at -5.01. SkillEvolver also improves more variants than it regresses (47 versus 20), whereas SPARK and CoEvoSkills show the reverse. The improvement and regression counts clarify the near-zero SPARK mean: it combines changes in opposite directions, rather than uniformly negligible effects. SkillEvolver shows both a larger positive mean and more frequent improvement than regression.

Replacing the instance produces the largest divergence. SkillEvolver remains positive under all three changes, with its largest gain on P2 ($+14.56$). P2 is also where SPARK and CoEvoSkills decline most (-5.23 and -9.51). SPARK retains positive averages under representation and requirement changes, but loses its advantage when the underlying instance changes. The P2 improvement–regression counts reinforce this divergence: 15 versus six for SkillEvolver, eight versus 14 for SPARK, and six versus 13 for CoEvoSkills. Replacing the instance changes both the average utility and the prevalence of helpful versus harmful changes. SkillEvolver's positive gains across P1–P3 show the broadest observed persistence of utility, while CoEvoSkills' negative means show that source-task guidance can become counterproductive after the task changes.

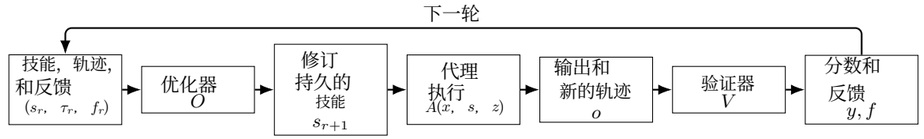


图1。技能进化的生命周期。构建产生一个持久的工件；部署将其指导转化为行为；验证分配观察到的分数。我们的评估通过配对结果、重复运行和冻结技能任务变化来衡量完成工件的效用。

2.2. 从分数收益到技能行为

分数 $V(x, A(x, s, z))$记录了执行者使用的技能的结果，并由验证器评估。将完成的技能与无技能进行比较，衡量其在部署时的效用。将采用的最终技能与其实际父技能进行比较，则解决了与修订相关的问题。这里的实验遵循第一种对比，即在重复运行和任务变化中进行比较。

对于每个源任务 x，我们冻结其完成的技能 s_x，并在一个变体 $T_k(x)$ 上与无技能执行进行评估。技能保持不变，而执行者可以重新解释输入，重新计算解决方案，并调整其动作。在变体上的正向配对差异表明，存储的指导在那种变化下仍然有用。变化类型区分了可以重用的决策和执行者必须重新考虑的决策。

为解释观察到的效用，我们考察其逐题分布。跨运行的一致性刻画同一批任务是否反复受益。分开统计正负分数变化，可以区分较小效应与正负抵消。按无技能分数分组，可区分零分任务上的帮助与满分执行上的损失。这些测量均以配对任务为分析单位，回答技能相关行为的不同问题。

3. 技能效用是否超出源任务?

我们首先测试完成的技能是否为其源任务提供反复的益处。然后我们改变任务，同时冻结其技能，以确定哪些需求该益处仍然存在。配对的无技能执行为两个比较提供了参考。

3.1. 实验设置

我们在 SkillsBench [4] 上评估 SkillEvolver [1]、SPARK [2] 和 CoEvoSkills [3]，每种方法运行两次，并与同题无技能执行配对。同一方法两次采用相同任务集：分别为81、84和85题。

我们报告0–100分制的平均验证器奖励及平均同题差值。对包含 N 个任务的运行，配对增益为

$$\Delta = \frac{100}{N} \sum_{i=1}^{N} \left(r_i^{\text{skill}} - r_i^{\text{no-skill}} \right). \tag{4}$$

我们还统计了具有正、负和零差异的任务数量。

3.2. 两次运行均有正向源任务效用

三种方法两次均有正向平均配对增益（表1）。SkillEvolver 为25.61和25.02分，CoEvoSkills 为10.24和9.76分，SPARK 为1.84和6.45分。

SkillEvolver 和 CoEvoSkills 的无技能分数虽变化较大，增益却相近。配对比较揭示了技能条件绝对分数可能掩盖的重复效用。

逐题结果也存在差异。SkillEvolver 每次的改进数至少为退化数的四倍；CoEvoSkills 两次均改进17题，分别退化9题和10题。SPARK 首次的改进与退化数接近，第二次为13次改进、5次退化。增益同时反映逐题变化的幅度和频率。冻结技能的变体评估检验源任务收益背后的指导能否持续适应不同需求。

3.3. 用任务变化检验哪些指导仍然有效

我们将每个源任务技能应用于无进一步进化的受控变体。输入表示变化（P1）会改变名称、列排列或模式；任务实例变化（P2）会替换数据或实例结构；耦合需求变化（P3）会改变或添加必须一起满足的需求。相应的决策要求是重新映射输入、为替换实例重新计算解决方案，并适应变化后的正确性要求。冻结的技能必须通过已存在于工件中的指导来支持这些适应性。

完整记录包含222个变体和跨越无技能和三种方法的886个条件级结果。表1总结了222个SkillEvolver变体的历史和恢复评估，以及SPARK和CoEvoSkills各221个变体的评估，将每种技能条件与同一变体上的无技能结果配对。

源任务改进不等于有效泛化。三种方法两次均提高源任务均分，变体结果却分化。SkillEvolver 保持11.94分的正向平均增益；SPARK 的 +0.65 接近零；CoEvoSkills 则低于无技能基线5.01分。SkillEvolver 的改进数多于退化数（47比20），另两种方法相反。SPARK 接近零的均值混合了方向相反的变化，并非所有影响均可忽略。SkillEvolver 同时具有更大的正向均值和更频繁的改进。

替换实例会产生最大的差异。 在三种变化下，SkillEvolver 均保持正值，其在 P2 上的增益最大 (+14.56)。P2 也是 SPARK 和 CoEvoSkills 下降最明显的地方 (-5.23 和 -9.51)。SPARK 在表示和需求变化下仍保持正值平均，但当底层实例变化时会失去优势。P2 的改进-退化计数强化了这种差异：SkillEvolver 为 15 对 6，SPARK 为 8 对 14，CoEvoSkills 为 6 对 13。替换实例会改变平均效用以及有益变化与有害变化的普遍性。SkillEvolver 在 P1–P3 上的正向增益显示了最广泛的效用持久性，而 CoEvoSkills 的负均值表明任务变化后，源任务指导可能变得适得其反。

Method	Evaluation	N	No skill	Skill	Δ	I	R	T	G	L
SkillEvolver	Source run 1	81	30.20	55.81	+25.61	27	6	48	30.91	5.30
	Source run 2	81	37.32	62.34	+25.02	26	3	52	28.32	3.30
	Variants: P1	77	42.04	53.89	+11.85	15	7	55	17.26	5.41
	Variants: P2	68	45.00	59.56	+14.56	15	6	47	22.06	7.50
	Variants: P3	77	38.94	48.65	+9.72	17	7	53	17.73	8.01
SPARK	Source run 1	84	17.82	19.66	+1.84	9	8	67	9.38	7.54
	Source run 2	84	14.37	20.81	+6.45	13	5	66	12.40	5.95
	Variants: P1	77	42.04	44.05	+2.00	12	11	54	13.43	11.43
	Variants: P2	68	45.00	39.77	−5.23	8	14	46	11.76	16.99
	Variants: P3	76	38.13	42.66	+4.53	12	12	52	15.79	11.26
CoEvoSkills	Source run 1	85	42.21	52.44	+10.24	17	9	59	17.14	6.90
	Source run 2	85	23.54	33.31	+9.76	17	10	58	18.26	8.50
	Variants: P1	77	42.04	38.46	−3.59	9	14	54	11.04	14.63
	Variants: P2	68	45.00	35.49	−9.51	6	13	49	8.09	17.60
	Variants: P3	76	38.13	35.71	−2.42	11	14	51	13.50	15.92

Baseline-conditioned variant outcomes

Method	Baseline zero	Zero to positive	Zero to full	Baseline full	Full to lower
SkillEvolver	126	42	40	86	16
SPARK	126	30	30	85	29
CoEvoSkills	126	25	23	85	32

Table 1. Completed-skill utility on source tasks and controlled variants. Rewards, paired gains Δ, positive contributions G, and negative contributions L use a 0–100 scale, with $\Delta = G - L$ before rounding. I/R/T counts improved, regressed, and tied tasks. P1: input representation; P2: task instance; P3: coupled requirements. Variant rows describe the complete historical and recovery record, pairing each skill result with its same-variant no-skill baseline. The lower panel counts transitions from zero or full baseline reward; each method also has ten intermediate-baseline variants.

4. WHAT PRODUCES THE NET SKILL GAIN?

We distinguish three contributors to mean utility: agreement on which tasks improve across runs, the balance between positive and negative changes, and whether the skill helps or harms execution at different no-skill reward levels.

4.1. Recurring Means Do Not Imply the Same Tasks Improve

Matching task identities across the two source-task runs reveals how much of the positive mean comes from recurring improvements. SkillEvolver improves 27 tasks in run 1 and 26 in run 2, but only 14 improve in both. The corresponding overlaps are three for SPARK and seven for CoEvoSkills. Across tasks that improve at least once, these counts are 14 of 39, three of 19, and seven of 27. Thus, the nearly unchanged mean gains of SkillEvolver and CoEvoSkills coexist with changes in which tasks benefit.

The transitions clarify this variation. Of SkillEvolver's 27 first-run improvements, 14 remain improvements, 12 become ties, and one becomes a regression. SPARK's nine first-run improvements become three improvements, four ties, and two regressions. CoEvoSkills' 17 become seven improvements, nine ties, and one regression. Losing an observed advantage most often means becoming tied with the no-skill execution, rather than reversing the sign. Conversely, ten, seven, and ten tasks move from a first-run tie to a second-run improvement for SkillEvolver, SPARK, and CoEvoSkills, respectively.

CoEvoSkills also regresses on five of the same tasks in both runs; neither other method has a task that regresses twice. This identifies a recurring source-task cost within a positive overall mean. The two runs therefore supply two distinct observations: each method has positive aggregate utility twice, while the task identities supporting that utility are only partly shared. These are comparisons of paired outcomes across runs; the shared object is the task, with each run using its recorded skill condition and execution.

4.2. Benefits and Losses Can Nearly Cancel

To distinguish small effects from cancellation, let $d_i = r_i^{\text{skill}} - r_i^{\text{no-skill}}$. We separate the variant mean into positive and negative contributions:

$$G = \frac{100}{N}\sum_i \max(d_i, 0), \quad L = \frac{100}{N}\sum_i \max(-d_i, 0). \tag{5}$$

The paired gain is $\Delta = G - L$. Both terms use all evaluated variants as their denominator, so they account for the frequency and magnitude of score changes together.

SkillEvolver contributes 18.89 points of improvement against 6.95 points of loss, leaving the observed +11.94 gain. SPARK contributes 13.73 positive points and 13.08 negative points. Its +0.65 mean therefore masks substantial movement in both directions. CoEvoSkills contributes 10.98 positive points, but its 15.98-point loss outweighs them. Its negative average includes useful changes on some variants alongside a larger cost elsewhere.

The conditional magnitudes further distinguish these patterns. Among improved variants, the average increase is 89.23 points for SkillEvolver, 94.82 for SPARK, and 93.30 for CoEvoSkills. Among regressed variants, the average loss is 77.16, 78.15, and 86.16 points. All three methods can produce large gains where they help. Their overall separation comes from how often those gains occur and how

方法	评估	N	无技能	技能	Δ	I	R	T	G	L
SkillEvolver	源运行1	81	30.20	55.81	+25.61	27	6	48	30.91	5.30
	源运行2	81	37.32	62.34	+25.02	26	3	52	28.32	3.30
	变体：P1	77	42.04	53.89	+11.85	15	7	55	17.26	5.41
	变体：P2	68	45.00	59.56	+14.56	15	6	47	22.06	7.50
	变体：P3	77	38.94	48.65	+9.72	17	7	53	17.73	8.01
SPARK	源运行1	84	17.82	19.66	+1.84	9	8	67	9.38	7.54
	源运行2	84	14.37	20.81	+6.45	13	5	66	12.40	5.95
	变体：P1	77	42.04	44.05	+2.00	12	11	54	13.43	11.43
	变体：P2	68	45.00	39.77	−5.23	8	14	46	11.76	16.99
	变体：P3	76	38.13	42.66	+4.53	12	12	52	15.79	11.26
CoEvoSkills	源运行1	85	42.21	52.44	+10.24	17	9	59	17.14	6.90
	源运行2	85	23.54	33.31	+9.76	17	10	58	18.26	8.50
	变体：P1	77	42.04	38.46	−3.59	9	14	54	11.04	14.63
	变体：P2	68	45.00	35.49	−9.51	6	13	49	8.09	17.60
	变体：P3	76	38.13	35.71	−2.42	11	14	51	13.50	15.92

按无技能基线分组的变体结果

方法	无技能零分	零分→正分	零分→满分	无技能满分	满分→降分
SkillEvolver	126	42	40	86	16
SPARK	126	30	30	85	29
CoEvoSkills	126	25	23	85	32

表1. 最终技能在源任务与受控变体上的效用。奖励、配对增益 Δ、正向贡献 G 与负向贡献 L 均按0–100分制报告，未四舍五入前满足 Δ=G −L。I/R/T 分别为改进、退化和持平数。P1为输入表示变化，P2为任务实例变化，P3为耦合需求变化。变体结果描述性汇总历史评估与补跑，每个技能结果均与同一变体的无技能结果配对。下半表统计从零分或满分基线出发的变化；每种方法另有10个基线得分介于两者之间的变体。

4. 净技能增益从何而来?

我们区分平均效用的三个组成因素：跨运行中受益任务的一致性、正负变化的平衡，以及技能在不同无技能得分水平下对执行结果的帮助或损失。

4.1. 均值反复为正，不代表同一批任务受益

匹配两次源任务运行中的任务身份，可检验正向均值有多少来自重复改进。SkillEvolver 两次分别改进27题和26题，但只有14题两次均改进；SPARK 和 CoEvoSkills 的重合数分别为3和7。在至少改进一次的任务中，两次均改进的比例分别为14/39、3/19和7/27。因此，SkillEvolver 和 CoEvoSkills 几乎不变的平均增益，也伴随着受益任务的变化。

逐题转变进一步解释了这种变化。SkillEvolver 首次改进的27题中，14题仍改进，12题变为持平，1题变为退化。SPARK 的9题分别变为3题改进、4题持平、2题退化；CoEvoSkills 的17题分别变为7题改进、9题持平、1题退化。失去原有优势，多数表现为与无技能执行持平，较少反转为负向差值。反过来，三种方法分别有10、7、10题由首次持平变为第二次改进。

CoEvoSkills 有5题两次均退化，另两种方法没有。这揭示了正向总体均值下反复出现的源任务损失。两次运行提供了两种不同的观察：各方法的总体效用两次均为正，但支撑收益的任务只部分重合。这是两次运行中配对结果的比较，共同对象是任务；每次运行采用其记录的技能条件和执行结果。

4.2. 收益与损失可能近乎抵消

令 d_i 为技能与无技能奖励之差。为区分较小效应与正负抵消，我们将变体均值分解为正、负两项：

$$G = \frac{100}{N}\sum_i \max(d_i, 0), \quad L = \frac{100}{N}\sum_i \max(-d_i, 0). \tag{5}$$

配对增益是 $\Delta = G - L$。这两个项都使用所有评估的变体作为它们的分母，因此它们共同考虑了分数变化的频率和幅度。

SkillEvolver 带来了 18.89 点的改进，抵消了 6.95 点的损失，留下了观察到的 +11.94 增益。SPARK 贡献了 13.73 点的正面影响和 13.08 点的负面影响。其 +0.65 平均值掩盖了两个方向上的显著波动。Co- EvoSkills 贡献了 10.98 点的正面影响，但其 15.98 点的损失超过了这些收益。其负面平均值包含了在某些变体上的有用变化，但同时在其他地方产生了更大的成本。

条件幅度进一步区分了这些模式。在改进的变体中，SkillEvolver的平均增幅为89.23分，SPARK为94.82分，CoEvoSkills为93.30分。在退化的变体中，平均损失分别为77.16分、78.15分和86.16分。这三种方法在它们发挥作用的地方都能产生巨大收益。它们的整体差异在于这些收益发生的频率以及

much is lost on other variants, rather than SkillEvolver alone producing large individual improvements. Its 47 improvements and 20 regressions give positive changes more weight; the balance reverses for SPARK (32 versus 37) and CoEvoSkills (26 versus 41).

4.3. Helping Zero-Reward Tasks and Preserving Full-Reward Tasks

Conditioning on the no-skill reward gives the changes a concrete interpretation. Each method's variant set contains 126 zero-reward no-skill executions. SkillEvolver raises 42 of them to positive reward, including 40 to full reward. SPARK raises 30 to full reward, and CoEvoSkills raises 25 to positive reward, including 23 to full reward. The respective rates of moving beyond zero are 33.3%, 23.8%, and 19.8%. SkillEvolver's advantage thus includes more frequent gains on variants where the executor receives no verifier credit without the skill.

The other endpoint measures losses on variants with a full-reward no-skill execution. Among SkillEvolver's 86 full-reward no-skill variants, 16 fall below full reward, including 15 that fall to zero. SPARK lowers 29 of 85, including 27 to zero; CoEvoSkills lowers 32 of 85, all to zero. The rates of losing full reward are 18.6%, 34.1%, and 37.6%. Each method also has ten intermediate-baseline variants, which remain included in the aggregate results but are outside these endpoint counts.

These two directions explain why counting newly successful tasks alone is insufficient. SPARK and CoEvoSkills both turn many zero-reward cases into full-reward outcomes, yet lose full reward on almost as many or more baseline successes. SkillEvolver combines the highest rate of gains from zero with the lowest rate of losses from full reward. Its broader utility is expressed in both acquiring a benefit and preserving outcomes that the executor already achieves.

5. INTERPRETING GAINS AS EVIDENCE OF SKILL LEARNING

5.1. Useful Guidance Must Survive a Change in Demand

Source-task utility establishes whether a completed skill helps on the task from which it was constructed. Frozen-skill variants test whether its guidance remains useful when original details no longer apply. Their three demands—remapping inputs, recomputing instance-dependent choices, and accommodating coupled requirements—probe different scopes of useful behavior.

This distinction is central to the observed method profiles. SPARK retains positive means under P1 and P3 but becomes negative under P2. Its source-task benefit therefore does not extend uniformly to replacement instances. CoEvoSkills has positive source-task means but negative means under every variant type. SkillEvolver combines positive means with more improvements than regressions in each type. The latter profile is the broadest evidence of useful generalization in these evaluations: the frozen guidance remains beneficial across different demands on the executor.

The published construction procedures suggest where these demands enter skill learning. SkillEvolver contrasts successful and failed deployments and audits reliance on training-instance constants [1]; SPARK distills environment-verified execution trajectories [2]; CoEvoSkills uses a co-evolving surrogate verifier to supply detailed revision feedback [3]. These mechanisms draw on different evidence when retaining guidance. Our behavioral results make abstraction from the source instance a concrete target for explaining their differences, especially the divergence under P2. Testing that explanation would require changing the construction feedback while holding the evaluation fixed.

5.2. Learning a Useful Skill Includes Avoiding Interference

The endpoint analysis adds a second requirement to generalization: a skill should help when the executor struggles without displacing decisions it already makes correctly. The no-skill condition makes both directions observable. Moving from zero to positive reward indicates additional useful behavior in the recorded execution. Falling from full reward identifies a lower-scoring skill execution paired with a full-reward no-skill execution. Reporting only the former would conceal a substantial part of the effect.

The SPARK and CoEvoSkills results show why this distinction changes the interpretation of learning. Both produce many large improvements, so their weak or negative variant means cannot be explained by an absence of helpful skills. Instead, their gains are offset by losses elsewhere. SkillEvolver's stronger net utility combines more gains from zero with fewer disruptions of full-reward outcomes. The diagnostic target is therefore the distribution of useful guidance: where it adds value, where it remains unnecessary, and where it interferes with execution.

6. RELATED WORK

Learning persistent skills from experience. SkillEvolver, SPARK, and CoEvoSkills make experience persistent through different construction loops: deployment-grounded revision, verified-trajectory distillation, and co-evolutionary verification, respectively [1–3]. These methods motivate our focus on the artifact delivered to a later executor. We study how that artifact's usefulness changes after its source task is varied, and analyze the task-level benefits and losses that produce its overall utility.

Evaluating skill utility and use. SkillsBench evaluates skill utility across diverse tasks, while SWE-Skills-Bench studies skill use in software engineering [4, 5]. SkillLearnBench evaluates continual skill generation on real-world tasks [6]. From Raw Experience to Skill Consumption separates experience generation, skill extraction, and downstream consumption [7]. Our study connects these evaluation levels to the identification of skill learning: repeated outcomes characterize completed-skill utility, task changes test its persistence, and score-transition analyses distinguish added benefits from losses of existing success. The unit of interpretation is the persistent artifact in use.

7. CONCLUSION

Our diagnosis examines what score gains from self-evolving skills represent beyond the source task. All three methods improve source-task means in two runs, but the individual tasks benefiting from them only partly recur. Frozen-skill variants expose different scopes of generalization, with SkillEvolver sustaining positive utility across representation, instance, and requirement changes. Decomposing the results shows that all three methods can deliver large individual gains; their net utility depends on the frequency of those gains and the losses incurred elsewhere. SkillEvolver combines more frequent gains from zero reward with fewer disruptions of full-reward executions. These findings connect the learning question to observable behavior: reusable guidance should add useful capability, retain its value when the task changes, and preserve success that the executor already achieves.

其他变体上损失多少，而非只有 SkillEvolver 能带来较大逐题改进。其47次改进与20次退化使正向变化占优；SPARK（32比37）与 CoEvoSkills（26比41）的关系则相反。

4.3. 帮助零分任务，保留满分结果

按无技能奖励分组，可具体解释这些变化。每种方法的变体集均有126次零分无技能执行。SkillEvolver 使其中42个获得正分，包括40个满分；SPARK 有30个变为满分；CoEvoSkills 有25个获得正分，包括23个满分。由零分转为正分的比例分别为33.3%、23.8%和19.8%。SkillEvolver 的优势包括：在无技能执行未获验证器奖励的变体上，更频繁地取得收益。

另一端点刻画无技能满分变体上的损失。SkillEvolver 的86个此类变体中，16个降分，15个降为零；SPARK 的85个中，29个降分，27个降为零；CoEvoSkills 的85个中，32个降分且均降为零。丢失满分的比例分别为18.6%、34.1%和37.6%。每种方法另有10个中间基线变体，计入总体结果，但不计入这两个端点。

这两个方向解释了为什么仅计算新成功的任务是不够的。SPARK 和 CoEvoSkills 都将许多零奖励情况转化为全奖励结果，但在几乎相同数量或更多基线成功的情况下却失去了全奖励。SkillEvolver 结合了从零奖励中获得最高收益率与从全奖励中损失最低的比率。它的更广泛实用性体现在既获取收益又保持执行者已实现的成果上。

5. 解释收益作为技能学习证据

5.1. 有用指导必须经受需求变化

源任务效用确立了完成的技能是否有助于其构建来源的任务。冻结技能变体测试当原始细节不再适用时其指导是否仍然有用。它们的三个需求——重新映射输入、重新计算实例依赖的选择、以及适应耦合需求——探测了不同范围的有用行为。

这一区别是观察到的方法特征的核心。SPARK 在 P1 和 P3 下保持正值但在 P2 下变为负值。因此其源任务收益并不均匀地扩展到替换实例。CoEvoSkills 具有正值源任务均值但在每种变体类型下均值均为负值。SkillEvolver 结合了正值均值与每种类型中比回归更多的改进。后者特征是这些评估中最广泛的泛化有用证据：冻结的指导在不同执行者需求下仍然有益。

各方法发表的构建流程提示了这些需求与技能学习的联系。SkillEvolver 对照成功和失败的部署，并检查对训练实例常量的依赖 [1]；SPARK 蒸馏经环境验证的执行轨迹 [2]；CoEvoSkills 通过协同进化的替代验证器提供详细修订反馈 [3]。这些机制依赖不同的证据来保留指导。行为结果使“从源实例中抽象指导”成为解释差异的具体方向，尤其是 P2 上的分化。检验这种解释，需要保持评估不变，仅调整构建反馈。

5.2. 学习一项有用技能包括避免干扰

端点分析为泛化增加了一个要求：技能应帮助遇到困难的执行者，同时保留其已有的正确决策。无技能条件使两种方向均可观测。从零分转为正分，表示记录中出现了额外的有用行为；从满分下降，则表示技能执行的分数低于其配对的满分无技能执行。只报告前一种变化，会掩盖效应的重要组成部分。

SPARK 和 CoEvoSkills 结果显示了为什么这种区分会改变对学习的理解。两者都产生了许多显著的改进，因此它们弱化或负面的变体不能归因于缺乏有用的技能。相反，它们的收益被其他地方的损失所抵消。SkillEvolver 更强的净效用结合了更多从零开始的收益，以及更少的对全奖励结果的干扰。因此，诊断目标是有用指导的分布：它在哪里增加价值，在哪里保持不必要，以及在哪里干扰执行。

6. 相关工作

从经验中学习持久的技能。 SkillEvolver、SPARK 和 CoEvoSkills 通过不同的构建循环使经验持久的：部署导向的修订、验证轨迹的蒸馏和协同进化的验证，分别 [1–3]。这些方法促使我们关注交付给后续执行者的工件。我们研究该工件的有用性在其源任务变化后的变化，并分析产生其整体效用的任务级收益和损失。

评估技能效用和使用。 SkillsBench 在多样化任务中评估技能效用，而 SWE-Skills-Bench 研究软件工程中的技能使用 [4, 5]。SkillLearnBench 评估在现实世界任务上的持续技能生成 [6]。从原始经验到技能消费将经验生成、技能提取和下游消费分开 [7]。我们的研究将这些评估级别与技能学习的识别联系起来：重复的结果表征完成技能的效用，任务变化测试其持久性，分数转换分析区分了现有成功的收益和损失。解释的单位是正在使用的持久工件。

7. 结论

我们的诊断考察了自我进化技能带来的分数提升在源任务之外的意义。所有三种方法在两次运行中提升了源任务的均值，但从中受益的个别任务仅部分重现。冻结技能的变体暴露了不同的泛化范围，而SkillEvolver在表示、实例和需求变化时仍保持正向效用。分解结果表明，所有三种方法都能带来显著的个体增益；它们的净效用取决于这些增益的频率以及在其他地方造成的损失。SkillEvolver结合了从零奖励中获得更频繁的增益，同时减少了全奖励执行的中断。这些发现将学习问题与可观察行为联系起来：可重用的指导应增加有用能力，在任务变化时保持其价值，并保留执行者已取得的成功。

8. REFERENCES

[1] Genrui Zhang, Erle Zhu, Jinfeng Zhou, Caiyan Jia, and Hongning Wang, "SkillEvolver: Skill Learning as a Meta-Skill," 2026.

[2] Yang Zhou, Zihan Dong, Zhenting Wang, Can Jin, Shiyu Zhao, Bangwei Guo, Difei Gu, Linjun Zhang, Mu Zhou, and Dimitris N. Metaxas, "Evidence Over Plans: Online Trajectory Verification for Skill Distillation," 2026.

[3] Hanrong Zhang, Shicheng Fan, Henry Peng Zou, Yankai Chen, Zhenting Wang, Jiayu Zhou, Chengze Li, Wei-Chieh Huang, Yifei Yao, Kening Zheng, Xue Liu, Xiaoxiao Li, and Philip S. Yu, "CoEvoSkills: Self-Evolving Agent Skills via Co-Evolutionary Verification," 2026.

[4] Xiangyi Li, Yimin Liu, Wenbo Chen, Bingran You, Zonglin Di, Yifeng He, Shenghan Zheng, Kyoung Whan Choe, Jiankai Sun, Shuyi Wang, Chujun Tao, Binxu Li, Xuandong Zhao, Hejia Geng, Xiaojun Wu, Junwei Zhou, Xiaokun Chen, Hanwen Xing, Yubo Li, Qunhong Zeng, Di Wang, Yuanli Wang, Roey Ben Chaim, Penghao Jiang, Haotian Shen, Luyang Kong, Xinyi Liu, Runhui Wang, Xuanqing Liu, Jiachen Li, Xin Lan, Yueqian Lin, Wengao Ye, Junwei He, Songlin Li, Yue Zhang, Yipeng Gao, Yijiang Li, Ze Ma, Liqiang Jing, Tianyu Wang, Kaixin Li, Yiqi Xue, Haoran Lyu, Yizhuo He, Yuchen Tian, Shutong Wu, Bowei Wang, Yixuan Gao, Bo Chen, Litong Liu, Sikai Cheng, Jiajun Bao, Shuaicheng Tong, Shuwen Xu, Terry Yue Zhuo, Tinghan Ye, Qi Qi, Miao Li, Longtai Liao, Zelin Tan, Chang Shi, Xilin Tang, Srinath Tankasala, Boqin Yuan, Yaoyao Qian, Jianhong Tu, Chenguang Wang, Yizhou Sun, Wei Wang, Aaron Taylor, Ziyue Yang, Changkun Guan, Zhikang Dong, Xinyu Zhang, Steven Dillmann, Han chung Lee, and Dawn Song, "SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks," 2026.

[5] Tingxu Han, Yi Zhang, Wei Song, Chunrong Fang, Zhenyu Chen, Youcheng Sun, and Lijie Hu, "SWE-Skills-Bench: Do Agent Skills Actually Help in Real-World Software Engineering?," 2026.

[6] Shanshan Zhong, Yi Lu, Jingjie Ning, Yibing Wan, Lihan Feng, Yuyi Ao, Leonardo F. R. Ribeiro, Markus Dreyer, Sean Ammirati, and Chenyan Xiong, "SkillLearnBench: Benchmarking Continual Learning Methods for Agent Skill Generation on Real-World Tasks," 2026.

[7] Zisu Huang, Jingwen Xu, Yifan Yang, Ziyang Gong, Qihao Yang, Muzhao Tian, Xiaohua Wang, Changze Lv, Xuemei Gao, Qi Dai, Bei Liu, Kai Qiu, Xue Yang, Dongdong Chen, Xiaoqing Zheng, and Chong Luo, "From Raw Experience to Skill Consumption: A Systematic Study of Model-Generated Agent Skills," 2026.

8. REFERENCES

[1] Genrui Zhang, Erle Zhu, Jinfeng Zhou, Caiyan Jia, and Hongning Wang, "SkillEvolver: Skill Learning as a Meta-Skill," 2026.

[2] Yang Zhou, Zihan Dong, Zhenting Wang, Can Jin, Shiyu Zhao, Bangwei Guo, Difei Gu, Linjun Zhang, Mu Zhou, and Dimitris N. Metaxas, "Evidence Over Plans: Online Trajectory Verification for Skill Distillation," 2026.

[3] Hanrong Zhang, Shicheng Fan, Henry Peng Zou, Yankai Chen, Zhenting Wang, Jiayu Zhou, Chengze Li, Wei-Chieh Huang, Yifei Yao, Kening Zheng, Xue Liu, Xiaoxiao Li, and Philip S. Yu, "CoEvoSkills: Self-Evolving Agent Skills via Co-Evolutionary Verification," 2026.

[4] Xiangyi Li, Yimin Liu, Wenbo Chen, Bingran You, Zonglin Di, Yifeng He, Shenghan Zheng, Kyoung Whan Choe, Jiankai Sun, Shuyi Wang, Chujun Tao, Binxu Li, Xuandong Zhao, Hejia Geng, Xiaojun Wu, Junwei Zhou, Xiaokun Chen, Hanwen Xing, Yubo Li, Qunhong Zeng, Di Wang, Yuanli Wang, Roey Ben Chaim, Penghao Jiang, Haotian Shen, Luyang Kong, Xinyi Liu, Runhui Wang, Xuanqing Liu, Jiachen Li, Xin Lan, Yueqian Lin, Wengao Ye, Junwei He, Songlin Li, Yue Zhang, Yipeng Gao, Yijiang Li, Ze Ma, Liqiang Jing, Tianyu Wang, Kaixin Li, Yiqi Xue, Haoran Lyu, Yizhuo He, Yuchen Tian, Shutong Wu, Bowei Wang, Yixuan Gao, Bo Chen, Litong Liu, Sikai Cheng, Jiajun Bao, Shuaicheng Tong, Shuwen Xu, Terry Yue Zhuo, Tinghan Ye, Qi Qi, Miao Li, Longtai Liao, Zelin Tan, Chang Shi, Xilin Tang, Srinath Tankasala, Boqin Yuan, Yaoyao Qian, Jianhong Tu, Chenguang Wang, Yizhou Sun, Wei Wang, Aaron Taylor, Ziyue Yang, Changkun Guan, Zhikang Dong, Xinyu Zhang, Steven Dillmann, Han chung Lee, and Dawn Song, "SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks," 2026.

[5] Tingxu Han, Yi Zhang, Wei Song, Chunrong Fang, Zhenyu Chen, Youcheng Sun, and Lijie Hu, "SWE-Skills-Bench: Do Agent Skills Actually Help in Real-World Software Engineering?," 2026.

[6] Shanshan Zhong, Yi Lu, Jingjie Ning, Yibing Wan, Lihan Feng, Yuyi Ao, Leonardo F. R. Ribeiro, Markus Dreyer, Sean Ammirati, and Chenyan Xiong, "SkillLearnBench: Benchmarking Continual Learning Methods for Agent Skill Generation on Real-World Tasks," 2026.

[7] Zisu Huang, Jingwen Xu, Yifan Yang, Ziyang Gong, Qihao Yang, Muzhao Tian, Xiaohua Wang, Changze Lv, Xuemei Gao, Qi Dai, Bei Liu, Kai Qiu, Xue Yang, Dongdong Chen, Xiaoqing Zheng, and Chong Luo, "From Raw Experience to Skill Consumption: A Systematic Study of Model-Generated Agent Skills," 2026.